

Лекция 7

Функции в языке Си

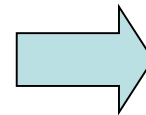
1 Объявление функций

Функция – это именованная часть программы, которая может быть многократно вызвана из другого участка программы.

```
Тип_Возвращаемого_Значения Имя_Функции( Формальные_Аргументы )  
{  
    Тело_Функции;  
    ...  
    return Возвращаемое_Значение;  
}
```

```
1  #include <stdio.h>  
2  
3  int sqr(int x)  
4  {  
5      int rez = x*x;  
6      return rez;  
7  }  
8
```

```
9  int main()  
10 {  
11     int n = 4;  
12     int r = sqr(n);  
13     printf("%d\n", r);  
14  
15     int x = 5;  
16     int rez = sqr(x) + 10;  
17     printf("%d\n", rez);  
18  
19     printf("%d", sqr(6));  
20     return 0;  
21 }
```



```
16  
35  
36
```

```

1  #include <stdio.h>
2
3  int F1(int a, int b)
4  {
5      int c = a+b;
6      return c;
7  }
8
9  float F2(float a, float b)
10 {
11     return a+b;
12 }
13
14 char* F3(char c)
15 {
16     c = c-32;
17     char* p=&c;
18     return p;
19 }
20

```

```

21 void F4(void)
22 {
23     printf("Hello, world!");
24 }
25
26 int main()
27 {
28     int i = F1(1,3);
29     printf("1) %d, %d\n", i, F1(2,5));
30
31     float f = F2(1.2,3.3);
32     printf("2) %.2f, %.2f\n", f, F2(2.5,5.5));
33
34     printf("3) %c\n", *( F3('x') ));
35     F4();
36
37     return 0;
38 }

```



```

1) 4, 7
2) 4.50, 8.00
3) X
Hello, world!

```

Поиск наибольшего из двух чисел:

```
1  #include <stdio.h>
2
3  // Способ 1
4  int max1(int a, int b)
5  {
6      int max;
7      if (a>b) max = a;
8      else max = b;
9      return max;
10 }
11
12 // Способ 2
13 int max2(int a, int b)
14 {
15     if (a>b) return a;
16     else return b;
17 }
18
19 // Способ 3
20 max3(int a, int b)
21 {
22     if (a>b) return a;
23     return b;
24 }
25
26 int main()
27 {
28     printf("%d\n", max1(10,20) );
29     printf("%d\n", max2(20,10) );
30     printf("%d",   max3(10,20) );
31     return 0;
32 }
```

20

20

20

Поиск в строке первого пробела и возврат адреса расположения первого пробела:

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <malloc.h>
4
5  char* find(char *str)
6  {
7      int i=0;
8      while (str[i] != '\0')
9      {
10         if (str[i] == ' ') return &str[i];
11         i++;
12     }
13     return NULL;
14 }
15
16 int main()
17 {
18     char stroka1[] = "One Two Three";
19     char *p = find(stroka1);
20     printf ("1) %ld\n", p - stroka1);
21
22     char *stroka2;
23     stroka2 = (char*)malloc(20*sizeof(char));
24     strcpy(stroka2,"Hello, world !");
25     p = find(stroka2);
26     printf ("2) %ld\n", p - stroka2);
27 }
```

1) 3

2) 6

2 Формальные и фактические параметры

Формальные параметры указываются в круглых скобках при объявлении функции.

Фактические параметры указываются при вызове функции.

```
1  #include <stdio.h>
2
3  int sum(int a, int b)
4  {
5      return a+b;
6  }
7
8  int main()
9  {
10     int s = sum(5, 6);
11     printf("1) %d\n", s);
12
13     int x=3, y=4;
14     s = sum(x, y);
15     printf("2) %d\n", s);
16
17     int a=1, b=2;
18     s = sum(a, b);
19     printf("3) %d\n", s);
20 }
```

a, b – формальные параметры

5, 6 – фактические параметры

x, y – фактические параметры

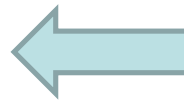
a, b – фактические параметры

3 Локальные и глобальные переменные

Локальная переменная – переменная, объявленная внутри блока, в частности внутри функции.

Локальная переменная доступна внутри блока, в котором она объявлена. При выходе из блока переменная удаляется.

```
m1 = 1, m2 = 2
f = 10
m1 = 1
```



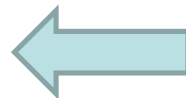
```
1  #include <stdio.h>
2
3  void fun(void)
4  {
5      int f = 10;
6      printf("f = %d\n", f);
7
8      // ОШИБКА, переменной m1 не существует
9      //printf("m1 = %d\n", m1);
10 }
11
12 int main()
13 {
14     int m1 = 1;
15
16     if (m1>0)
17     {
18         int m2 = 2;
19         printf("m1 = %d, m2 = %d\n", m1, m2);
20
21         fun();
22
23         // ОШИБКА, переменной f не существует
24         //printf("f = %d\n", f);
25     }
26
27     // ОШИБКА, переменной m2 не существует
28     //printf("m1 = %d, m2 = %d\n", m1, m2);
29
30     printf("m1 = %d\n", m1);
31 }
```

Глобальная переменная

– переменная,
объявленная вне какой-
либо функции.

Глобальная переменная
может быть использована
в любом месте
программы после своего
объявления.

```
m1 = 1, m2 = 2
f = 10
f = 11
```



```
1  #include <stdio.h>
2
3  int f = 10;
4
5  void fun(void)
6  {
7      printf("f = %d\n", f);
8      f++;
9
10     // ОШИБКА, переменных m1 и m2 не существует
11     //printf("m1 = %d, m2 = %d\n", m1, m2);
12 }
13
14 int m1 = 1, m2 = 2;
15
16 int main()
17 {
18     if (m1>0)
19     {
20         printf("m1 = %d, m2 = %d\n", m1, m2);
21
22         fun();
23         printf("f = %d\n", f);
24
25         m2++;
26     }
27
28     printf("m1 = %d, m2 = %d\n", m1, m2);
29 }
```

4 Передача аргументов

Способ 1: передача по значению

В вызываемой функции для каждого формального аргумента создается локальный объект, который инициализируется значением фактического аргумента. При такой передаче изменения значений формальных параметров функции не приводят к изменению значений соответствующих им фактических аргументов.

```
1  #include <stdio.h>
2
3  void IncA(int a)
4  {
5      printf("IncA : a = %d\n", a);
6      ++a;
7      printf("IncA : ++a = %d\n", a);
8  }
9
10 int main()
11 {
12     int b = 5;
13     IncA(b);
14     printf("main : b = %d\n", b);
15 }
```

```
IncA : a = 5
IncA : ++a = 6
main : b = 5
```

```
1  #include <stdio.h>
2
3  void IncA(int a)
4  {
5      printf("IncA : a = %d\n", a);
6      ++a;
7      printf("IncA : ++a = %d\n", a);
8  }
9
10 int main()
11 {
12     int a = 5;
13     IncA(a);
14     printf("main : a = %d\n", a);
15 }
```

```
IncA : a = 5
IncA : ++a = 6
main : a = 5
```

Способ 2: передача по указателю

При таком способе внутри функции создается локальная переменная указатель. Поскольку этот указатель инициализируется адресом переменной из вызываемой подпрограммы, то значение переменной будет изменяться.

```
1  #include <stdio.h>
2
3  void IncA(int *a)
4  {
5      printf("IncA : a = %d\n", *a);
6      *a = *a + 1;
7      printf("IncA : ++a = %d\n", *a);
8  }
9
10 int main()
11 {
12     int b = 5;
13     IncA(&b);
14     printf("main : b = %d\n", b);
15 }
```



```
IncA : a = 5
IncA : ++a = 6
main : b = 6
```

```
1  #include <stdio.h>
2
3  void IncA(int *a)
4  {
5      printf("IncA : a = %d\n", *a);
6      *a = *a + 1;
7      printf("IncA : ++a = %d\n", *a);
8  }
9
10 int main()
11 {
12     int a = 5;
13     IncA(&a);
14     printf("main : a = %d\n", a);
15 }
```



```
IncA : a = 5
IncA : ++a = 6
main : a = 6
```

Для изменения значения переменную, необходимо передавать функции указатель на эту переменную

Чтобы изменить указатель, необходимо передавать указатель на указатель

```
1 #include <stdio.h>
2 #include <malloc.h>
3
4 void init(int *a, unsigned size)
5 {
6     a = (int*) malloc(size * sizeof(int));
7 }
8
9 int main()
10 {
11     int *a = NULL;
12     init(a, 100);
13
14     if (a == NULL) printf("ERROR");
15     else {
16         printf("Память выделена!");
17         free(a);
18     }
19 }
```



ERROR

```
1 #include <stdio.h>
2 #include <malloc.h>
3
4 void init(int **a, unsigned size)
5 {
6     *a = (int*) malloc(size * sizeof(int));
7 }
8
9 int main()
10 {
11     int *a = NULL;
12     init(&a, 100);
13
14     if (a == NULL) printf("ERROR");
15     else {
16         printf("Память выделена!");
17         free(a);
18     }
19 }
```



Память выделена!



```
1 #include <stdio.h>
2 #include <malloc.h>
3
4 int* init(int *a, unsigned size)
5 {
6     a = (int*) malloc(size * sizeof(int));
7     return a;
8 }
9
10 int main()
11 {
12     int *a;
13     a = init(a, 100);
14
15     if (a == NULL) printf("ERROR");
16     else {
17         printf("Память выделена!");
18         free(a);
19     }
20 }
```

5 Передача массива в функцию

```
1 #include <stdio.h>
2
3 // void printArray(int arr[10], unsigned size)
4 // void printArray(int arr[], unsigned size)
5 void printArray(int *arr, unsigned size)
6 {
7     unsigned i;
8     for (i = 0; i < size; i++)
9     {
10         printf("%d ", arr[i]);
11     }
12 }
13
14 int main()
15 {
16     int x[10] = {1, 2, 3, 4, 5};
17     printArray(x, 10);
18 }
```



1 2 3 4 5 0 0 0 0 0

1 2 3 4 5
6 7 8 9 10



```
1 #include <stdio.h>
2
3 //void printArray(int arr[2][5], unsigned size)
4 //void printArray(int arr[][5], unsigned size)
5 void printArray(int (*arr)[5], unsigned size)
6 {
7     unsigned i, j;
8     for (i = 0; i < size; i++)
9     {
10         for (j = 0; j < 5; j++)
11         {
12             printf("%d ", arr[i][j]);
13         }
14         printf("\n");
15     }
16 }
17
18 int main()
19 {
20     int x[][5] = { { 1, 2, 3, 4, 5},
21                   { 6, 7, 8, 9, 10} };
22     printArray(x, 2);
23 }
```

6 Прототип функции

Тип_Возвращ_Значения Имя_Функции(Параметры);

```
1  #include <stdio.h>
2
3  //int sum(int a, int b);
4  int sum(int, int);
5
6  int main()
7  {
8      int s = sum(3, 4);
9      printf("s = %d", s);
10 }
11
12 int sum(int a, int b)
13 {
14     return a + b;
15 }
```



s = 7

7 Рекурсивные функции

Рекурсия — это такая организация выполнения работы функции, при которой данная функция вызывает сама себя.

ВИДЫ РЕКУРСИИ

Прямая рекурсия

```
void f()
{
    .....
    f();
    .....
}
```

Косвенная рекурсия

```
void f2()

void f1()
{
    .....
    f2();
    .....
}

void f2()
{
    .....
    f1();
    .....
}
```

Обязательные действия при разработке рекурсивной функции:

- 1) изменение параметра;
- 2) условие остановки рекурсии.

Прямая рекурсия

```
1  #include <stdio.h>
2
3  void Rec(int a)
4  {
5      if (a>0) Rec(a-1);
6      printf("%d\n", a);
7  }
8
9  int main()
10 {
11     Rec(3);
12     return 0;
13 }
```



0
1
2
3

Процедура Rec(3)

Процедура Rec(2)

Процедура Rec(1)

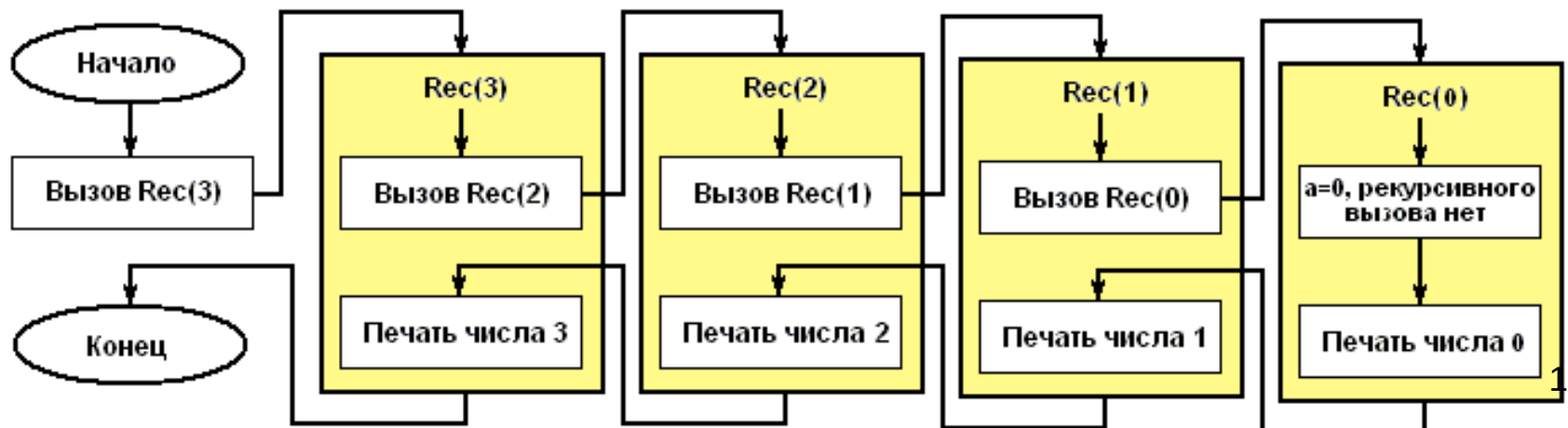
Процедура Rec(0)

Печать числа 0

Печать числа 1

Печать числа 2

Печать числа 3



Косвенная рекурсия

```
1  #include <stdio.h>
2
3  void Rec2(int);
4
5  void Rec1(int a)
6  {
7      if (a>0) Rec2(a-1);
8      printf("Rec1: %d\n", a);
9  }
10
11 void Rec2(int a)
12 {
13     Rec1(a-1);
14     printf("Rec2: %d\n", a);
15 }
16
17 int main()
18 {
19     Rec1(3);
20     return 0;
21 }
```



```
Rec1: -1
Rec2: 0
Rec1: 1
Rec2: 2
Rec1: 3
```

ФОРМЫ РЕКУРСИВНЫХ ПОДПРОГРАММ



Форма с выполнением действий до рекурсивного вызова (с выполнением действий на рекурсивном спуске)

```
void Rec()
{
    // действия

    if (условие) Rec();
}
```



Форма с выполнением действий как до, так и после рекурсивного вызова (с выполнением действий как на рекурсивном спуске так и на рекурсивном возврате)

```
void Rec()
{
    // действия

    if (условие) Rec();

    // действия
}
```



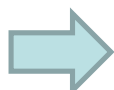
Форма с выполнением действий после рекурсивного вызова (с выполнением действий на рекурсивном возврате)

```
void Rec()
{
    if (условие) Rec();

    // действия
}
```

Рекурсивный спуск

```
1 #include <stdio.h>
2
3 void Rec(int a)
4 {
5     printf("%d\n", a);
6     if (a>0) Rec(a-1);
7 }
8
9 int main()
10 {
11     Rec(5);
12     return 0;
13 }
```



5
4
3
2
1
0

Рекурсивный возврат

```
1 #include <stdio.h>
2
3 void Rec(int a)
4 {
5     if (a>0) Rec(a-1);
6     printf("%d\n", a);
7 }
8
9 int main()
10 {
11     Rec(5);
12     return 0;
13 }
```



0
1
2
3
4
5

Рекурсивный спуск и возврат

```
1 #include <stdio.h>
2
3 void Rec(int a)
4 {
5     printf("%d\n", a);
6     if (a>0) Rec(a-1);
7     printf("%d\n", a);
8 }
9
10 int main()
11 {
12     Rec(5);
13     return 0;
14 }
```



5
4
3
2
1
0
0
1
2
3
4
5

Рекурсия и итерация (вычисление факториала)

```
1 #include <stdio.h>
2
3 int Fact(int n)
4 {
5     int F = 1, i;
6     for (int i=1; i<=n; i++)
7     {
8         F=F*i;
9     }
10    return F;
11 }
12
13 int main()
14 {
15     printf("%d", Fact(5));
16     return 0;
17 }
```

```
1 #include <stdio.h>
2
3 int Fact(int n)
4 {
5     if (n==1) return 1;
6     else return n*Fact(n-1);
7 }
8
9 int main()
10 {
11     printf("%d", Fact(5));
12     return 0;
13 }
```

120

Fact(5) → 5 * Fact(4) 5 * 24 → 120

↓

4 * Fact(3) 4 * 6

↓

3 * Fact(2) 3 * 2

↓

2 * Fact(1) 2 * 1

↓

1 → 1

Рекурсия и итерация (вывод элементов массива)

```
1 #include <stdio.h>
2
3 void printArray(int *arr, unsigned size)
4 {
5     for (int i = 0; i < size; i++)
6     {
7         printf("%d ", arr[i]);
8     }
9 }
10
11 int main()
12 {
13     int x[10] = {1, 2, 3, 4, 5};
14     printArray(x, 10);
15 }
```

```
1 #include <stdio.h>
2
3 void printArray(int *arr, unsigned size)
4 {
5     if (size != 1) printArray(arr, size-1);
6     printf("%d ", arr[size-1]);
7 }
8
9 int main()
10 {
11     int x[10] = {1, 2, 3, 4, 5};
12     printArray(x, 10);
13 }
```



1 2 3 4 5 0 0 0 0 0

Рекурсия и итерация (поиск элемента в массиве)

```
1 #include <stdio.h>
2
3 int Find(int *arr, int size, int N)
4 {
5     for (int i = 0; i < size; i++)
6     {
7         if (arr[i] == N) return i;
8     }
9     return -1;
10 }
11
12 int main()
13 {
14     int x[7] = {6, 8, 2, 5, 3, 4, 1};
15
16     // массив x из 7 элементов; ищем 3
17     int i = Find(x, 7, 3);
18
19     if (i >= 0) printf("x[%d] = 3", i);
20     else printf("Элемент не найден!");
21 }
```

```
1 #include <stdio.h>
2
3 int Find(int *arr, int size, int N)
4 {
5     if (arr[size-1] == N) return size-1;
6     else
7     if (size == 0) return -1;
8     Find(arr, size-1, N);
9 }
10
11 int main()
12 {
13     int x[7] = {6, 8, 2, 5, 3, 4, 1};
14
15     // массив x из 7 элементов; ищем 3
16     int i = Find(x, 7, 3);
17
18     if (i >= 0) printf("x[%d] = 3", i);
19     else printf("Элемент не найден!");
20 }
```

x[4] = 3